

LISTITERATOR

¿Qué es?

- Iterator

Un Iterator es una forma de recorrer una colección elemento por elemento. Va avanzando uno a uno y solo puede ir hacia adelante.

- ListIterator

Un ListIterator solo se puede usar con listas.

Esto es porque en las listas los elementos tienen un orden numerado (0, 1, 2...).

- Puede avanzar hacia adelante
- Puede retroceder hacia atrás
- Permite modificar la lista mientras se recorre

Ejemplo para entenderlo mejor

- El Iterator es como un **pasajero** que solo mira hacia adelante.
- El ListIterator es como un **editor**, que tiene control total sobre la posición y el contenido de la lista.

La Posición

El cursor no está sobre un elemento, sino entre los elementos.

Si tienes una lista:

[A] [B] [C]

El cursor puede estar en 4 posiciones:

^ [A] [B] [C] → Antes del primer elemento

[A] ^ [B] [C] → Entre A y B

[A] [B] ^ [C] → Entre B y C

[A] [B] [C] ^ → Después del último elemento

Métodos que utiliza

add(E e)

→ ¿Qué hace?

Mete un elemento nuevo justo donde está el cursor en ese momento.

→ ¿Qué devuelve?

void → No devuelve nada. Inserta un elemento en la lista en la posición que este.

hasNext()

→ ¿Qué hace?

Pregunta: "¿Hay algo más adelante?".

→ ¿Qué devuelve?

boolean:

- true si hay un siguiente elemento
- false si ya estás en el final de la lista

hasPrevious()

→ ¿Qué hace?

Pregunta: "¿Hay algo detrás?". Se utiliza para saber si puedes retroceder en la lista.

→ ¿Qué devuelve?

boolean:

- true si hay un elemento anterior
- false si estás en el inicio

next()

→ ¿Qué hace?

Avanza al siguiente elemento.

→ ¿Qué devuelve?

E → Devuelve el siguiente elemento de la lista.

nextIndex()

→ ¿Qué hace?

Indica la posición del elemento que conseguiría con next().

→ ¿Qué devuelve?

int → Índice del siguiente elemento.

previous()

→ ¿Qué hace?

Retrocede al elemento anterior.

→ ¿Qué devuelve?

E → Devuelve el elemento anterior de la lista.

previousIndex()

→ ¿Qué hace?

Indica la posición del elemento que conseguiría con `previous()`.

→ ¿Qué devuelve?

int → Índice del elemento anterior.

remove()

→ ¿Qué hace?

Elimina de la lista el último elemento que ha devuelto `next()` o `previous()`.

→ ¿Qué devuelve?

void → No devuelve nada.

set(E e)

→ ¿Qué hace?

Reemplaza el último elemento conseguido por `next()` o `previous()` por uno nuevo.

→ ¿Qué devuelve?

void → No devuelve nada.

¿Qué sucede en una lista vacía?

Imagina que tienes una caja vacía. El cursor solo tiene una posición posible (inicio y fin coinciden).

- `hasNext()` → **false**, es false porque no hay elementos delante
- `hasPrevious()` → **false**, es false porque no hay elementos detrás
- `nextIndex()` → **0**, es porque el primer elemento iría en posición 0
- `previousIndex()` → **-1**, es porque no hay elemento anterior

Ejemplo

ENUNCIADO DEL PROYECTO

Una tienda de productos tecnológicos desea desarrollar una aplicación en Java para gestionar su catálogo de productos de forma eficiente.

La tienda necesita un sistema que permita a los empleados realizar las siguientes operaciones:

1. Añadir nuevos productos al catálogo.
2. Mostrar todos los productos almacenados en orden de inserción.
3. Mostrar los productos en orden inverso para facilitar revisiones rápidas.
4. Modificar los datos de un producto existente a partir de su ID.
5. Eliminar productos del catálogo cuando dejen de estar disponibles.
6. Insertar productos en una posición concreta de la lista (por ejemplo, para reorganizar el catálogo).

Para implementar estas funcionalidades, se utilizará una estructura de datos tipo lista (ArrayList), y su recorrido y manipulación se realizará mediante la interfaz ListIterator.

Clase Producto:

```
public class Producto {  
    private int id;  
    private String nombre;  
    private double precio;  
  
    public Producto(int id, String nombre, double precio) {  
        super();  
        this.id = id;  
        this.nombre = nombre;  
        this.precio = precio;  
    }  
    public int getId() {  
        return id;  
    }  
    public void setId(int id) {  
        this.id = id;  
    }  
    public String getNombre() {  
        return nombre;  
    }  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
    public double getPrecio() {  
        return precio;  
    }  
    public void setPrecio(double precio) {  
        this.precio = precio;  
    }  
    @Override  
    public String toString() {  
        return "Producto [id=" + id + ", nombre=" + nombre + ", precio=" + precio +  
    }  
}
```

Clase ProductoCRUD:

```
import java.util.ArrayList;
import java.util.ListIterator;
public class ProductoCRUD {
    private ArrayList<Producto> lista = new ArrayList<>();
    // CREATE
    public void agregarProducto(Producto p) {
        lista.add(p);
    }
    // READ
    public void mostrarProductos() {
        ListIterator<Producto> it = lista.listIterator();
        while (it.hasNext()) {
            System.out.println(it.next());
        }
    }
    // READ
    public void mostrarProductosReversa() {
        ListIterator<Producto> it = lista.listIterator(lista.size());
        while (it.hasPrevious()) {
            System.out.println(it.previous());
        }
    }
    // UPDATE
    public void modificarProducto(int id, String nuevoNombre, double nuevoPrecio)
{
    ListIterator<Producto> it = lista.listIterator();
    while (it.hasNext()) {
        Producto p = it.next();
        if (p.getId() == id) {
            p.setNombre(nuevoNombre);
            p.setPrecio(nuevoPrecio);
            // También se podría usar set()
            it.set(p);
            System.out.println("Producto modificado.");
            return;
        }
    }
    System.out.println("Producto no encontrado.");
}
    // DELETE
    public void eliminarProducto(int id) {
        ListIterator<Producto> it = lista.listIterator();
        while (it.hasNext()) {
            Producto p = it.next();
```



```
        if (p.getId() == id) {
            it.remove(); // clave del ejercicio
            System.out.println("Producto eliminado.");
            return;
        }
    }
    System.out.println("Producto no encontrado.");
}

// INSERTAR EN POSICIÓN
public void insertarEnPosicion(int posicion, Producto p) {
    ListIterator<Producto> it = lista.listIterator();
    int contador = 0;
    while (it.hasNext()) {
        if (contador == posicion) {
            it.add(p);
            System.out.println("Insertado en posición " + posicion);
            return;
        }
        it.next();
        contador++;
    }
    // Si no encuentra posición, lo añade al final
    lista.add(p);
}
}
```

Clase Principal:

```
public class Principal {
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        ProductoCRUD crud = new ProductoCRUD();
        int opcion, id, idMod, idEl, pos;
        String nombre, nuevoNombre;
        double precio, nuevoPrecio;
        do {
            System.out.println("\n--- MENU ---");
            System.out.println("1. Añadir producto");
            System.out.println("2. Mostrar productos");
            System.out.println("3. Mostrar productos (reversa)");
            System.out.println("4. Modificar producto");
            System.out.println("5. Eliminar producto");
            System.out.println("6. Insertar en posición");
            System.out.println("0. Salir");
            opcion = Leer.datoInt();
            switch (opcion) {
                case 1:
                    System.out.print("ID: ");
                    id = Leer.datoInt();
                    System.out.print("Nombre: ");
                    nombre = Leer.dato();
                    System.out.print("Precio: ");
                    precio = Leer.datoDouble();
                    crud.agregarProducto(new Producto(id, nombre, precio));
                    break;
                case 2:
                    crud.mostrarProductos();
                    break;
                case 3:
                    crud.mostrarProductosReversa();
                    break;
                case 4:
                    System.out.print("ID a modificar: ");
                    idMod = Leer.datoInt();
                    System.out.print("Nuevo nombre: ");
                    nuevoNombre = Leer.dato();
                    System.out.print("Nuevo precio: ");
                    nuevoPrecio = Leer.datoDouble();
                    crud.modificarProducto(idMod, nuevoNombre, nuevoPrecio);
                    break;
                case 5:
                    System.out.print("ID a eliminar: ");
                    idEl = Leer.datoInt();
```




```
        crud.eliminarProducto(idEI);  
        break;  
    case 6:  
        System.out.print("Posición: ");  
        pos = Leer.datoInt();  
        System.out.print("ID: ");  
        id = Leer.datoInt();  
        System.out.print("Nombre: ");  
        nombre = Leer.dato();  
        System.out.print("Precio: ");  
        precio = Leer.datoDouble();  
        crud.insertarEnPosicion(pos, new Producto(id, nombre, precio));  
        break;  
    }  
} while (opcion != 0);  
}  
}
```